

Introduction to Git and GitHub

*Shagun Kanani
Computer Science and Engineering
Formula Manipal - Driverless Taskphase
MIT, Manipal*

Abstract— This report provides a practical study of Git, a distributed version-control system, and GitHub, a platform for hosting and collaborating on Git repositories. Core concepts covered include the working directory, staging area, commits, branches, and Git's internal object model, along with operations such as merging, rebasing, conflict resolution, stashing, and remote synchronization through cloning, pushing, fetching, and pulling. Theoretical understanding was reinforced through the interactive "Learn Git Branching" simulator (both the Main and Remote modules) and applied directly to my task 1 repository containing python programs for Formula Manipal Driverless taskphase.

Keywords— *Git, GitHub, Commit, Branch, Push, Repositories, Commands, Main*

1. Introduction

Git is a distributed version-control system that tracks changes to files and maintains the complete history of a repository locally, on the developer's own machine. GitHub is a web-based platform that hosts Git repositories remotely and adds collaboration features such as pull requests, issue tracking, and access control. Git is the underlying tool that performs version control, while GitHub is one of several possible remote hosts for a Git repository.

This report documents my taskphase exercise to build a working understanding of Git that goes beyond simply uploading files to GitHub. The exercise combined three components: theoretical study of Git's model and commands, hands-on practice using the "Learn Git Branching" simulator, and a practical implementation carried out on my task 1 repository containing python programs.

2. Objectives

The main objective was to understand Git's version-control model and why it is needed, the working directory, staging area, repository, commits, branches, and HEAD. Also, to understand Git's commit/history model and its internal object storage. Learn remote-repository operations like clone, push, pull, and fetch and to understand merging, rebasing, merge conflicts, and conflict resolution.

Another objective was to complete both the Main and Remote sections of Learn Git Branching and demonstrate Git operations on my task 1 repository.

3. Git Fundamentals

A. What is Git?

Git is a distributed version-control system: every clone of a repository contains the full project history, not just a snapshot of the current files. Git tracks changes to files over time, stores that history in a repository, and allows multiple independent lines of development through branches.

Git operates locally by default; a remote repository, such as one hosted on GitHub, is used only when collaboration or backup is needed.

B. Git Repository

A repository is a directory that Git manages. Running `git init` inside a folder creates a hidden `.git` subdirectory, which contains everything Git needs to track history: objects, references, and configuration.

For this exercise, the Task One folder was initialized with `git init`. The resulting status showed no commits yet and listed the existing Task One files as untracked, confirming that Git was tracking the directory but had not yet recorded any history.

C. Working Directory

The working directory is the actual set of files on disk that make up the project. In this case, the Task One program folder (1.1, 1.2, and subsequent solutions). Editing any of these files modifies the working directory only; Git does not record a change until it is explicitly staged.

D. Staging Area

The staging area (or "index") sits between the working directory and the repository. It lets a developer choose exactly which changes will be included in the next commit, rather than committing every modified file automatically.

Relevant commands: `git status`, `git add <file>`, `git add`
In practice, running `git status` before staging showed the modified Task One file listed under "Changes not staged for commit"; after `git add "1.1 freq of char"`, the same file appeared under "Changes to be committed", confirming the transition into the staging area.

E. Commits

A commit is a saved snapshot of the staged changes, together with a message describing what changed and why. It is a small object that stores metadata, the author, a timestamp, a commit message, a reference to the parent commit, and a reference to a tree object that represents the complete directory structure at that point in time. Each commit is identified by a unique hash and, except for the first commit in a repository, points to a parent commit, forming a chain that constitutes the project's history. In this repository, the commit `git commit -m "Update 1.1"` produced the commit identified as `b6d02c5`, visible via `git log --oneline`.

F. Commit Hashes

Every Git object is identified by a hash computed from its contents and metadata, which guarantees that identical content always produces the same identifier and that any change produces a different one. Commands such as `git log --oneline` display a shortened form of this hash, such as `b6d02c5`, which is sufficient to uniquely identify the commit within the repository.

G. HEAD

HEAD is a reference that points to the current position in the repository, normally the tip of the branch that is currently checked out. The output

`b6d02c5 (HEAD -> main, origin/main, origin/HEAD)` indicates that HEAD is pointing at commit `b6d02c5` on the local main branch, and that this position also matches the remote-tracking reference for main on GitHub.

H. Branches

A branch is a lightweight, movable reference to a commit. Because a branch is just a pointer rather than a copy of the project, creating one is cheap, and multiple branches can develop independently before being reconciled through a merge or rebase.

Relevant commands: `git branch`, `git branch <name>`, `git switch <name>`, `git switch -c <name>`.

`git switch -c feature-branch` was used to create a branch in task 1.1.

4. Git History and Branch Operations

A. Log and Reflog

Git log lists the commits reachable from the current HEAD, forming the project's committed history; `git log --oneline --graph --all` condenses this into a single line per commit and draws the branch structure, which is especially useful once multiple branches exist. `git reflog`, by contrast, records every movement of HEAD and other references — including commits that are no longer reachable from any branch — and is therefore a valuable safety net for recovering from mistakes such as an accidental reset.

B. Merge

Merging combines the history of one branch into another. If the target branch has not moved since the source branch diverged, Git performs a fast-forward merge, simply moving the pointer forward. Otherwise, Git creates a dedicated merge commit with two parents, joining the two histories.

Relevant commands: `git switch main`, `git merge feature`.

C. Merge Conflicts

Git resolves most merges automatically, but a conflict arises when the same region of a file has been changed differently on both branches and Git cannot determine which version to keep. The conflicting file is marked with conflict markers, and the developer must edit the file to the intended final content before staging and committing the resolution.

D. Rebase

Rebasing replays the commits of one branch onto the tip of another, producing a linear history instead of a branching one. Because each replayed commit is technically a new object with a new hash, rebase rewrites history and should be used with care on any branch that has already been shared with others.

`git rebase -i` (interactive rebase) allows the user to modify a sequence of recent commits. It can be used to reorder commits, edit commit messages, combine commits using squash, or remove commits before the history is shared. For example, `git rebase -i HEAD~3` opens the last three commits for interactive editing.

E. Git Cherry-Pick

`git cherry-pick` applies the changes introduced by a specific existing commit onto the current branch. Instead of merging an entire branch, it allows a selected commit to be brought into another branch. For example, `git cherry-pick <commit-hash>` applies the changes from the specified commit to the current branch and creates a new commit.

F. Stash

`git stash` temporarily sets aside uncommitted changes in the working directory and staging area, restoring a clean working directory without requiring a commit. `git stash list` shows saved stashes; `git stash pop` restores the most recent stash and removes it from the stash list, while `git stash apply` restores it but keeps it in the list for reuse.

G. Force Push

A normal `git push` is rejected if the remote branch has commits that the local branch does not. `git push --force` overrides this check and overwrites the remote branch to match the local one, which can discard remote commits and is therefore risky on any branch other people rely on. `git push --force-with-lease` is a safer variant that fails if the remote has moved since it was last fetched, reducing the risk of silently discarding someone else's work.

5. Remote Repositories

A. Local vs. Remote Repositories

A remote is another copy of the repository, typically hosted elsewhere, that a local repository can synchronize with. In this project, the remote is the GitHub repository containing the Task One files.

B. Origin

origin is simply the conventional name Git assigns to the remote a repository was cloned from; it is not a special keyword, only a default label. Running `git remote -v` lists the configured remotes and their URLs.

C. Clone

`git init` creates a brand-new, empty local repository, whereas `git clone <URL>` copies an existing repository, including its entire commit history, from a remote location into a new local directory. Because the GitHub repository already contained the Task One files, `git clone` was the operation used to bring that history onto the local machine.

D. Push

`git push` uploads local commits that do not yet exist on the remote, bringing the remote branch up to date with the local one. After pushing, `git log --oneline` showed `b6d02c5 (HEAD -> main, origin/main, origin/HEAD)`, confirming that the local main branch, the remote-tracking reference, and the actual GitHub branch were all aligned at the same commit.

E. Fetch

`git fetch` downloads any new commits and references from the remote but does not merge them into the current branch; it only updates the local remote-tracking references, such as `origin/main`, leaving the working branch untouched until the developer chooses to integrate the changes.

F. Pull

`git pull` is equivalent to a fetch followed immediately by a merge (or rebase, depending on configuration) of the fetched changes into the current branch, bringing the local branch up to date with the remote in a single step.

G. Remote-Tracking Branches

`main` is the local branch that is actively worked on; `origin/main` is not the branch stored on GitHub itself, but a local reference that records where the remote's main branch was as of the last fetch or push. Understanding this distinction clarifies why `origin/main` can briefly disagree with the true state of GitHub until a fetch or push refreshes it.

6. Practical Implementation

Cloning the GitHub repository

The existing GitHub repository, which already contained the Task One files, was cloned locally using `git clone <GitHub-repository>`.

Modifying Task One

The "1.1 freq of char" program was edited and `git status` subsequently reported it as modified.

Staging and committing the change

The modified file was staged with `git add "1.1 freq of char"` and committed with `git commit -m "Update 1.1"`, producing commit b6d02c5 (confirmed with `git log --oneline`).

Pushing to GitHub

`git push` uploaded the new commit; the subsequent log entry b6d02c5 (HEAD -> main, origin/main, origin/HEAD) confirmed that the local and remote branches were synchronized.

7. Branching, Merging, and Conflict-Resolution Demonstration

A. Creating and switching branches

A new branch was created by `git switch -c feature-branch`. `git branch` was used to confirm the currently active branch.

B. Committing on the practice branch

A change was made and committed on the feature branch using `git add "1.1 freq of char"`. and `git commit -m "Update 1.1 on feature branch"`. Switching back to main with `git switch main` and re-inspecting the file confirmed that main was unaffected by the change. then `git push -u origin feature-branch` to create remote branch.

C. Merging

From main, `git merge feature-branch` merged the feature branch's commit into main. The merge resulted in a fast-forward because main had not diverged from feature-branch.

D. Rebasing

On a separate practice branch, `git log --oneline --graph --all` was recorded before rebasing to capture the starting history. `git rebase main` was then run to replay the practice branch's commits onto the current tip of main, and the same log command was run again afterward to show the resulting linear history in place of the earlier branching structure.

8. Learn Git Branching

A. Main

The Main section of the Learn Git Branching simulator was completed. Its levels build up from basic commits and branch creation to interactive rebasing and cherry-picking, and were especially useful for visualizing the commit graph in a way that a plain terminal does not make obvious.

B. Remote

The Remote section was also completed. It focuses on push, fetch, and pull between a simulated local and remote repository, and on the behavior of remote-tracking branches such as origin/main. This reinforced the distinction between a local branch and the local record of a remote branch's state.

9. Conclusion

This taskphase exercise built a working understanding of Git as a distributed version-control system and of GitHub as a platform for hosting and synchronizing Git repositories, going beyond simple file uploads to cover the staging area, the commit and object model, branching, merging, conflict resolution, rebasing, stashing, and remote operations such as cloning, pushing, fetching, and pulling.

Theoretical study was reinforced through the Learn Git Branching simulator, and applied directly to the Task One repository, where the repository was initialized, staged, committed, and pushed to GitHub, and further exercised through dedicated branching, merging, conflict, and rebase demonstrations.

Together, the theory and the practical work show how Git's layered model of local commits and remote-tracking references supports a development workflow that is both traceable and safe to collaborate on.

10. References

- [1] "Learn Git Branching," learngitbranching.js.org. [Online]. Available: <https://learngitbranching.js.org/>
- [2] GitHub Docs, "About Git." [Online]. Available: <https://docs.github.com/en/get-started/using-git/about-git>
- [3] Video resource provided with the taskphase assignment.